

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

**Факультет Информационных технологий
Кафедра Информатики и информационных технологий**

направление подготовки

09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 12

Дисциплина: «BackEnd-разработка»

Тема: Обработка ошибок в веб-приложении на основе ASP.NET Core

Выполнил(а): студент(ка) группы 241-333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись

(Дата)

(Подпись)

Проверил:

(Фамилия И.О., степень, звание)

(Оценка)

Дата, подпись

(Дата)

(Подпись)

**Москва
2026**

Цель работы:

Ознакомиться с основными способами обработки ошибок в веб-приложениях на платформе ASP.NET Core.

Задание:

1. Создать проект ASP.NET Core Web API или использовать ранее созданный проект.
2. Реализовать обработку ошибок бизнес-логики с использованием ModelState.
3. Реализовать фильтр для обработки ошибок, возникающих при работе с базой данных.
4. Настроить глобальную обработку необработанных исключений в приложении.
5. Протестировать обработку различных типов ошибок через Swagger или Postman.

1. Написание приложения

1. **Использование существующего проекта.** В качестве основы для работы использован проект StoreApi из лабораторной работы №11, реализующий REST API для интернет-магазина. Проект построен на платформе ASP.NET Core 10 с использованием Entity Framework Core и базы данных MySQL. Схема базы данных включает пять связанных сущностей: Category, Product, Customer, Order, OrderItem.
2. **Архитектура обработки ошибок.** В проекте реализован многоуровневый подход к обработке ошибок: Валидация входных данных, Обработка бизнес-ошибок, Фильтр исключений БД и Глобальный обработчик.
3. **Валидация через ModelState и DTO.** Все DTO-классы используют атрибуты System.ComponentModel.DataAnnotations для валидации:

```
public record CreateProductDto(  
    [Required, MaxLength(100)] string Name,  
    [Range(0.01, 999999.99)] decimal Price,  
    [Range(0, int.MaxValue)] int Stock,  
    [Range(1, int.MaxValue)] int CategoryId  
);
```

В контроллерах проверка выполняется автоматически:

```

if (!ModelState.IsValid)
    return BadRequest(new ProblemDetails
    {
        Status = 400,
        Title = "Validation failed",
        Detail = "One or more validation errors occurred.",
        Instance = HttpContext.Request.Path,
        Extensions = { ["errors"] = ... }
    });

```

4. **Фильтр исключений для работы с базой данных.** Создан класс DatabaseExceptionHandler, реализующий интерфейс IExceptionHandler, который перехватывает DbUpdateException и преобразует технические ошибки БД в понятные клиенту ответы:

```

public class DatabaseExceptionHandler : IExceptionHandler
{
    public void OnException(ExceptionContext context)
    {
        if (context.Exception is DbUpdateException dbEx)
        {
            var inner = dbEx.InnerException?.Message?.ToLower();

            if (inner?.Contains("foreign key constraint fails") == true)
            {
                context.Result = new ObjectResult(new ProblemDetails
                {
                    Status = 409,
                    Title = "Referential integrity violation",
                    Detail = "Cannot modify record: it is referenced by other data.",
                    Instance = context.HttpContext.Request.Path
                })
                { StatusCode = 409 };
                context.ExceptionHandled = true;
            }
            // ... обработка других ошибок БД
        }
    }
}

```

5. **Глобальная обработка необработанных исключений.** Для перехвата всех неперехваченных ошибок используется UseExceptionHandler.
6. **Интеграция в контроллеры.** Все контроллеры (CategoriesController, ProductsController, CustomersController, OrdersController) обновлены для возврата ошибок в едином формате ProblemDetails. Пример обработки бизнес-ошибки в OrdersController:

```

if (product.Stock < itemDto.Quantity)
    return BadRequest(new ProblemDetails
    {
        Status = 400,
        Title = "Insufficient stock",
        Detail = $"Not enough stock for \"{product.Name}\". Available: {product.Stock}, requested: {itemDto.Quantity}",
        Instance = HttpContext.Request.Path
    });

```

2. Запуск приложения и тестирование

Для тестирования функционала API использовался Swagger UI.

Таблица 1 Реализованные механизмы обработки ошибок

Механизм	Реализация	Статус-коды
Валидация ModelState	Атрибуты в DTO + проверка в контроллерах	400 Bad Request
Бизнес-ошибки	Проверка условий перед сохранением	400, 404
Ошибки БД	DatabaseExceptionHandler : IExceptionHandler	409 Conflict
Глобальные ошибки	UseExceptionHandler + ProblemDetails	500 Internal Server Error

Таблица 2 Результаты тестирования ключевых сценариев

№	URL	Метод	Параметры	Результат
1	/api/Products	POST	{"name":"","price":-10,"stock":1,"categoryId":1}	400
2	/api/Orders	POST	{"customerId":99999,"items":[...]}	404
3	/api/Orders	POST	{"customerId":1,"items":[{"productId":1,"quantity":9999}]}	400
4	/api/Categories/1	DELETE	- (есть связанные товары)	409
5	/api/test-crash	GET	- (искусственное исключение)	500

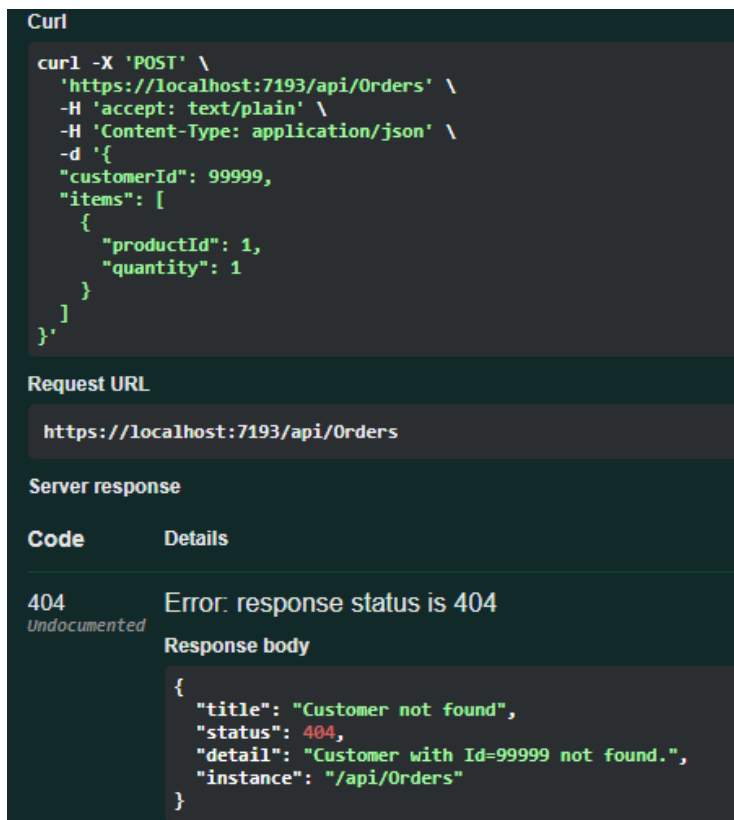
The screenshot displays the Swagger UI interface for a REST API. It shows a 'Curl' section with a POST request to 'https://localhost:7193/api/Products'. Below this, the 'Request URL' is confirmed as 'https://localhost:7193/api/Products'. The 'Server response' section indicates a '400' status code with the message 'Error: response status is 400'. The 'Response body' is a JSON object detailing the validation errors: the 'Name' field is required, and the 'Price' field must be between 0.01 and 999999.99. A 'traceId' is also provided for logging purposes.

```
Curl
curl -X 'POST' \
  'https://localhost:7193/api/Products' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{"name":"","price":-10,"stock":1,"categoryId":1}'

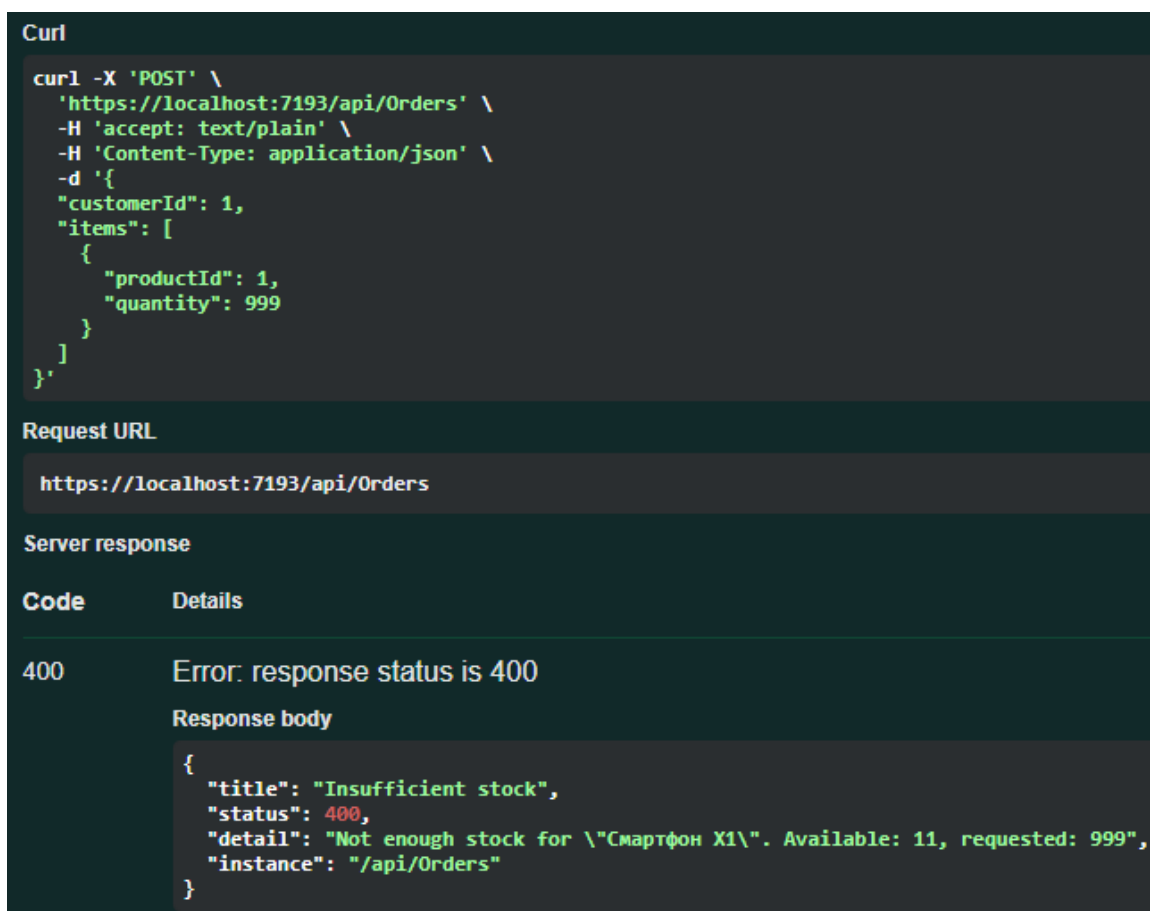
Request URL
https://localhost:7193/api/Products

Server response
Code      Details
400       Error: response status is 400
Response body
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": [
      "The Name field is required."
    ],
    "Price": [
      "The field Price must be between 0,01 and 999999,99."
    ]
  },
  "traceId": "00-95feb183f2c5a9bdad5b833b46d7676b-96aeaa91ffc1786b-00"
}
```

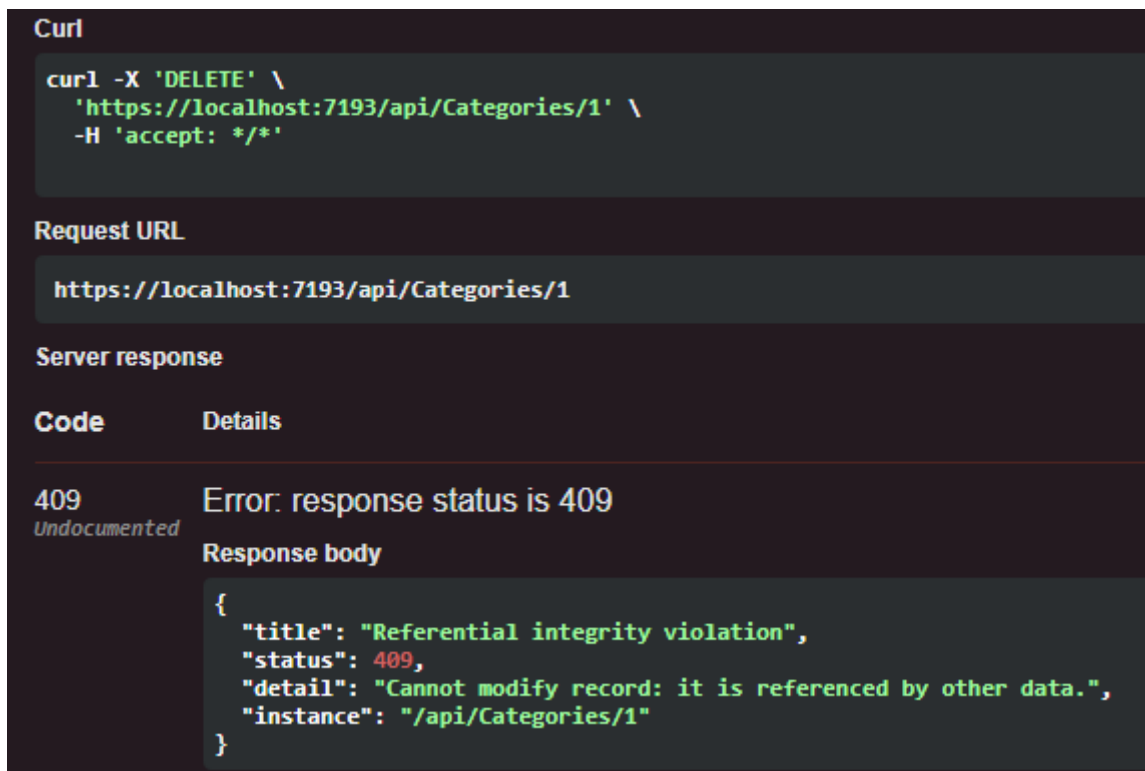
Изображение 1 код 400 POST /api/Products



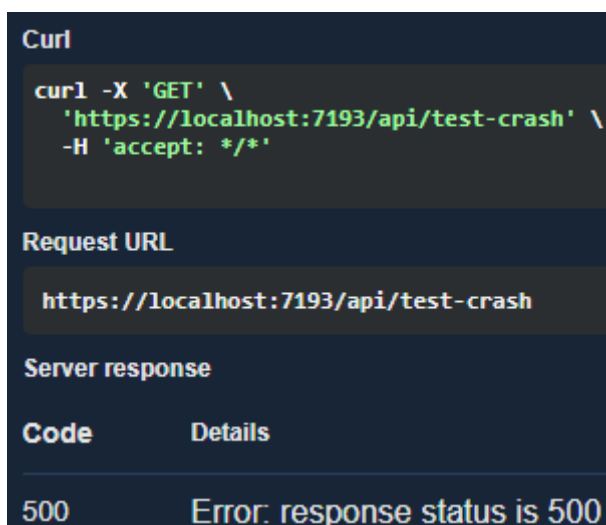
Изображение 2 код 404 POST /api/Orders



Изображение 3 код 400 POST /api/Orders



Изображение 4 код 409 DELETE /api/Categories/1



Изображение 5 код 500 GET /api/test-crash

3. Листинг кода

Program.cs

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using StoreApiLR11.Filters;

builder.Services.AddControllers(options =>
{
    // Регистрация фильтра для обработки ошибок БД
    options.Filters.Add<DatabaseExceptionHandler>();
});

var app = builder.Build();

// Глобальный обработчик – ПЕРВЫМ в конвейере
if (app.Environment.IsDevelopment())
    app.UseExceptionHandler("/error-development");
else
    app.UseExceptionHandler("/error");

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

// Эндпоинт для обработки ошибок (стандартизированный ответ)
app.Map("/error", async (HttpContext context) =>
{
    var exception = context.Features.Get<ExceptionHandlerPathFeature>()?.Error;
    context.Response.StatusCode = 500;
    context.Response.ContentType = "application/problem+json";

    await context.Response.WriteAsJsonAsync(new ProblemDetails
    {
        Status = 500,
        Title = "Internal server error",
        Detail = app.Environment.IsDevelopment()
            ? exception?.Message
            : "An unexpected error occurred.",
        Instance = context.Request.Path
    });
});

app.Run();
```

Фильтр исключений БД (Filters/DatabaseExceptionHandler.cs)

```

public class DatabaseExceptionHandler : IExceptionHandler
{
    private readonly ILogger<DatabaseExceptionHandler> _logger;

    public DatabaseExceptionHandler(ILogger<DatabaseExceptionHandler> logger)
    {
        _logger = logger;
    }

    public void OnException(ExceptionContext context)
    {
        if (context.Exception is DbUpdateException dbEx)
        {
            _logger.LogError(dbEx, "Database update error");

            var inner = dbEx.InnerException?.Message?.ToLower();

            // Нарушение внешнего ключа
            if (inner?.Contains("foreign key constraint fails") == true)
            {
                context.Result = new ObjectResult(new ProblemDetails
                {
                    Status = 409,
                    Title = "Referential integrity violation",
                    Detail = "Cannot modify record: it is referenced by other data.",
                    Instance = context.HttpContext.Request.Path
                })
                { StatusCode = 409 };
                context.ExceptionHandled = true;
                return;
            }

            // Дубликат уникального поля
            if (inner?.Contains("duplicate entry") == true ||
                inner?.Contains("unique constraint") == true)
            {
                context.Result = new ObjectResult(new ProblemDetails
                {
                    Status = 409,
                    Title = "Duplicate entry",
                    Detail = "A record with the same unique value already exists.",
                    Instance = context.HttpContext.Request.Path
                })
                { StatusCode = 409 };
                context.ExceptionHandled = true;
                return;
            }

            // Остальные ошибки БД -> 500
            context.Result = new ObjectResult(new ProblemDetails
            {
                Status = 500,
                Title = "Database error",
                Detail = "An error occurred while saving data.",
                Instance = context.HttpContext.Request.Path
            })
            { StatusCode = 500 };
            context.ExceptionHandled = true;
        }
    }
}

```

Единый формат ошибок в контроллерах (пример из OrdersController)

```

// Бизнес-ошибка: недостаточно товара
using Microsoft.AspNetCore.Mvc;
using StoreApiLR11.Models;

if (product.Stock < itemDto.Quantity)
    return BadRequest(new ProblemDetails
    {
        Status = 400,
        Title = "Insufficient stock",
        Detail = $"Not enough stock for \"{product.Name}\". Available: {product.Stock},
requested: {itemDto.Quantity}",
        Instance = HttpContext.Request.Path
    });

// Валидация ModelState
if (!ModelState.IsValid)
    return BadRequest(new ProblemDetails
    {
        Status = 400,
        Title = "Validation failed",
        Detail = "One or more validation errors occurred.",
        Instance = HttpContext.Request.Path,
        Extensions = { ["errors"] = ... }
    });

```